

# Computer Vision Practical Report:

## DRAW RECTANGULAR SHAPE AND EXTRACT OBJECT

### 1. AIM

To manually select a Region of Interest (ROI) by drawing a rectangular bounding shape on an image using interactive mouse callbacks or bounding coordinates in OpenCV, extract the specified object region, display the cropped object separately, and save the result.

### 2. PROCEDURE

- Load the source image using `cv2.imread()`.
- Interactively select a rectangular region using `cv2.selectROI()` (or bind mouse click events with `cv2.setMouseCallback()` to capture coordinates  $(x, y, w, h)$ ).
- Draw the visible rectangular boundary on the original image using `cv2.rectangle()`.
- Extract the enclosed object region using NumPy array slicing (`extracted_object = image[y:y+h, x:x+w]`).
- Display the original image with the drawn rectangle alongside the extracted object using Matplotlib or `cv2.imshow()`, and save the cropped object using `cv2.imwrite()`.

### 3. PROGRAM CODE

```
import cv2
import matplotlib.pyplot as plt

# 1. Load image
image = cv2.imread('E40-IMG.png')

if image is None:
    print("Error: Image not found.")
else:
    # 2. Select ROI by dragging mouse (Press ENTER or SPACE to confirm)
```

```
print("Select rectangle using mouse, then press ENTER or SPACE.")
r = cv2.selectROI("Draw Rectangle to Extract", image)
cv2.destroyAllWindows()

x, y, w, h = int(r[0]), int(r[1]), int(r[2]), int(r[3])

if w > 0 and h > 0:
    # 3. Crop object using NumPy slicing
    extracted_object = image[y:y+h, x:x+w]

    # 4. Draw bounding rectangle on original copy
    output_image = image.copy()
    cv2.rectangle(output_image, (x, y), (x + w, y + h), (0, 255, 0), 3)

    # Convert to RGB for Matplotlib display
    output_rgb = cv2.cvtColor(output_image, cv2.COLOR_BGR2RGB)
    extracted_rgb = cv2.cvtColor(extracted_object, cv2.COLOR_BGR2RGB)

    # 5. Display side-by-side on same page
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(output_rgb)
    plt.title("Drawn Rectangle ROI")
    plt.axis("off")

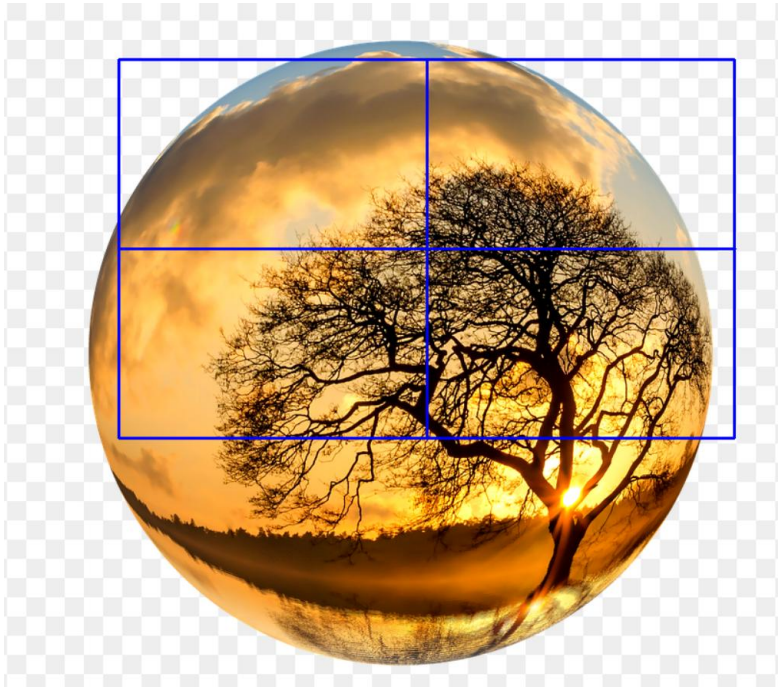
    plt.subplot(1, 2, 2)
    plt.imshow(extracted_rgb)
    plt.title("Extracted Object")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Save cropped result
```

```
cv2.imwrite("extracted_manual.png", extracted_object)
```

#### 4. OUTPUT



#### 5. RESULT

The Python script executed successfully. The input image was loaded.